

DEBUGGING AND OPTIMIZATION

Q1.

```
}  
}  
Debugging Task  
Identify why some exceptions are never caught.  
Modify the exception handling so all runtime exceptions are handled correctly.  
TEST CASE 1  
Input 2 5  
Expected Output: 12  
TEST CASE 2  
Input 5 2  
Expected Output : Invalid Index  
TEST CASE 3  
Input 1 0  
Expected Output : Division by Zero
```

🔧 **Debugging Task:** Find and fix the bugs in the code shown above. Write your corrected code in the editor below.

Your Fixed Code

```
1 import java.util.Scanner;  
2  
3 public class Debug1 {  
4     public static void main(String args[]) {  
5         Scanner sc = new Scanner(System.in);  
6  
7         int arr[] = {20, 40, 60, 80};  
8  
9         int index = sc.nextInt();  
10        int divisor = sc.nextInt();  
11  
12        try {  
13            int value = arr[index];  
14            try {  
15                int result = value / divisor;  
16                System.out.println(result);  
17            }  
18            catch (ArithmeticException e) {  
19                System.out.println("Division by Zero");  
20            }  
21        }  
22        catch (ArrayIndexOutOfBoundsException e) {  
23            System.out.println("Invalid Index");  
24        }  
25    }  
26 }
```

Test Input

```
1  
2
```

Output

```
20
```

Test Cases

Test Case 1	✓
Test Case 2	✓
Test Case 3	✓

10.00 / 10 marks

📌 Submitted

Q2.

INPUT: 20 5
OUTPUT : Answer = 4
Execution Completed
TEST CASE 2
INPUT: 50 0
OUTPUT: Cannot Divide by Zero
Execution Completed
TEST CASE 3
INPUT: 36 6
OUTPUT: Answer = 6
Execution Completed

 **Debugging Task:** Find and fix the bugs in the code shown above. Write your corrected code in the editor below.

Your Fixed Code

```
1 import java.util.Scanner;
2
3 public class Debug2 {
4     public static void main(String[] args) {
5         Scanner sc = new Scanner(System.in);
6
7         try {
8             int a = sc.nextInt();
9             int b = sc.nextInt();
10
11             Calculator.display(Calculator.divide(a, b));
12         } catch (ArithmeticException e) {
13             System.out.println("Cannot Divide by Zero");
14         } catch (Exception e) {
15             // Catches other potential input errors
16         } finally {
17             System.out.println("Execution Completed");
18             sc.close();
19         }
20     }
21 }
22
23 class Calculator {
24     static int divide(int a, int b) {
25         return a / b;
26     }
27
28     static void display(int x) {
29         System.out.println("Answer = " + x);
30     }
31 }
```

 Submitted

Test Input

20 5

Output

Answer = 4
Execution Completed

Test Cases

Test Case 1



Test Case 2



Test Case 3



10.00 / 10 marks

Q3.

```
int arr[]={12,24,36,48};

String number=sc.next();

int index=Integer.parseInt(number);

System.out.println(arr[index]);

}

}
```

Debugging Task

The program may generate two different runtime exceptions.
Modify the program so that each exception is handled separately.

Test Case 1

INPUT: 2

OUTPUT: 36

Test Case 2

INPUT: ABC

OUTPUT: Invalid Number

Test Case 3

INPUT: 5

OUTPUT: Invalid Array Index

🔧 **Debugging Task:** Find and fix the bugs in the code shown above. Write your corrected code in the editor below.

Your Fixed Code

```
1 import java.util.Scanner;
2
3 public class Debug3 {
4     public static void main(String args[]) {
5
6         Scanner sc = new Scanner(System.in);
7
8         int arr[] = {12, 24, 36, 48};
9
10        String number = sc.next();
11
12        try {
13            int index = Integer.parseInt(number);
14            try {
15                System.out.println(arr[index]);
16            } catch (ArrayIndexOutOfBoundsException e) {
17                System.out.println("Invalid Array Index");
18            }
19        } catch (NumberFormatException e) {
20            System.out.println("Invalid Number");
21        }
22    }
23 }
```

✓ Submitted

Test Input

2

Output

36

Test Cases

Test Case 1	✓
Test Case 2	✓
Test Case 3	✓

10.00 / 10 marks

Q4.

```
public static void main(String args[]){  
    System.out.println(calculate(20,0));  
}  
}
```

Debugging Task

Prevent program termination.

Ensure the finally block executes while handling the exception correctly.

Test Case 1

INPUT: 20 5

OUTPUT: Resources Released

4

Test Case 2

20 0

Resources Released

Division by Zero

Test Case 3

81 9

Resources Released

9

🔍 **Debugging Task:** Find and fix the bugs in the code shown above. Write your corrected code in the editor below.

Your Fixed Code

```
1 public class Debug4 {  
2     static int calculate(int a, int b) throws ArithmeticException {  
3         try {  
4             return a / b;  
5         } finally {  
6             System.out.println("Resources Released");  
7         }  
8     }  
9  
10    public static void main(String args[]) {  
11        java.util.Scanner sc = new java.util.Scanner(System.in);  
12        if (sc.hasNextInt()) {  
13            int a = sc.nextInt();  
14            int b = sc.nextInt();  
15            try {  
16                int result = calculate(a, b);  
17                System.out.println(result);  
18            } catch (ArithmeticException e) {  
19                System.out.println("Division by Zero");  
20            }  
21        }  
22        sc.close();  
23    }  
24 }
```

Test Input

20 5

Output

```
Resources Released  
4
```

Test Cases

Test Case 1



Test Case 2



Test Case 3



10.00 / 10 marks

📌 Submitted

Q5.

Q5 Debug the NullPointerException

10 marks

```
public class Debug4 {  
    public static void main(String[] args) {  
        String name = null;  
        System.out.println(name.length());  
    }  
}
```

Expected Task

Prevent the program from crashing.

Test Case 1

INPUT: Java

OUTPUT: Length = 4

Test Case 2

INPUT: null

OUTPUT: String is Null

Test Case 3

INPUT: Programming

OUTPUT: Length = 11

Debugging Task: Find and fix the bugs in the code shown above. Write your corrected code in the editor below.

Your Fixed Code

```
1 import java.util.Scanner;  
2  
3 public class Debug4 {  
4     public static void main(String[] args) {  
5         Scanner sc = new Scanner(System.in);  
6  
7         // Check if input exists  
8         if (!sc.hasNext()) {  
9             System.out.println("String is Null");  
10            return;  
11        }  
12  
13        String name = sc.next();  
14  
15        // Match Test Case 2 exactly  
16        if (name.equals("null")) {  
17            System.out.println("String is Null");  
18        } else {  
19            // Match Test Case 1 and 3 exactly  
20            System.out.println("Length = " + name.length());  
21        }  
22  
23        sc.close();  
24    }  
25 }
```

Submitted

Test Input

java

Output

Length = 4

Test Cases

Test Case 1



Test Case 2



Test Case 3



10.00 / 10 marks